

Redis

peanutixx

2026 年 6 月 20 日

Contents

1 简介	1
2 安装	2
3 Redis 常用命令	2
3.1 基础命令	2
3.2 操作键 (Key) 的命令	4
3.3 字符串	4
3.4 列表	6
3.5 哈希	6
3.6 集合	8
3.7 有序集合	9
4 Redis 事务	11
4.1 相关命令	11
4.2 执行流程	11
4.3 Redis 事务的 ACID 特性（与关系型数据库对比）	12
5 Redis 的持久化机制	12
5.1 RDB 快照	12
5.2 AOF 日志	14
6 缓存问题：缓存穿透、缓存击穿、缓存雪崩	15
6.1 缓存穿透	15
6.2 缓存击穿	17
6.3 缓存雪崩	17
7 Redis++	18
7.1 安装	18
7.2 示例代码	19

1 简介

[Redis](#) (Remote Dictionary Server) 是一款开源的高性能的键值对存储系统，通常被称为远程数据结构服务器。Redis 支持丰富的数据类型，如：字符串 (string)、哈希 (hash)、列表 (list)、集合 (set)、有序集合 (zset)、...

Redis 使用 C 语言开发，遵守 BSD 协议，是互联网技术中使用最为广泛的中间件之一。随着它在新浪微博等大型互联网公司的使用而逐渐风靡全国，也是后端开发工程师面试的热点。

2 安装

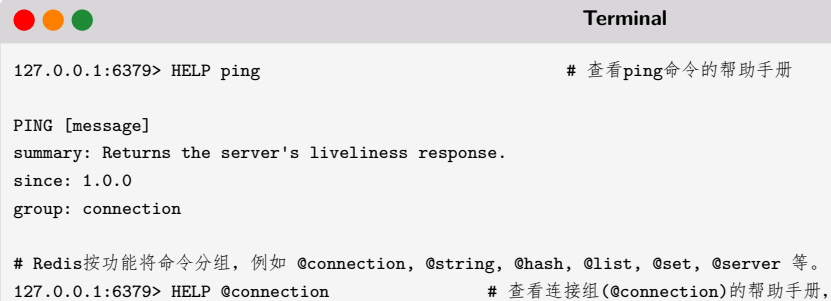
```
1 # 更新本地软件包索引, 获取最新的可用软件包列表
2 $ sudo apt update
3
4 # 安装 Redis 服务器, -y(yes) 参数自动确认所有提示
5 $ sudo apt install redis -y
6
7 # 查看已安装的 Redis 服务器版本信息
8 # 输出示例: v=6.0.16, 使用 jemalloc 内存分配器, 64 位架构
9 $ redis-server -v
10 Redis server v=6.0.16 sha=00000000:0 malloc=jemalloc-5.2.1 bits=64 build=e91d7986ac6d5bb6
11
12 # 将当前用户添加到 redis 用户组, 使其获得 redis 组权限
13 # (需要重新登录才能生效)
14 $ sudo usermod -aG redis $USER
15
16 退出当前终端会话, 重新登录
17
18 # 为 redis 配置文件增加组写入权限, 允许 redis 组成员修改配置
19 $ sudo chmod g+w /etc/redis/redis.conf
20
21 # 递归地为 Redis 数据目录 (/var/lib/redis) 增加组写入权限
22 # 允许 redis 组成员在该目录下创建/修改/删除文件
23 $ sudo chmod -R g+w /var/lib/redis
```

3 Redis 常用命令

3.1 基础命令

查看帮助手册

我们可以在 `redis-cli` 终端输入 `HELP <命令名>` 查看 Redis 内置的帮助手册。



```
Terminal
127.0.0.1:6379> HELP ping # 查看ping命令的帮助手册

PING [message]
summary: Returns the server's liveliness response.
since: 1.0.0
group: connection

# Redis按功能将命令分组, 例如 @connection, @string, @hash, @list, @set, @server 等。
127.0.0.1:6379> HELP @connection # 查看连接组(@connection)的帮助手册,
```

Redis 官方还提供了详尽的技术文档, 可供进一步查阅:

- [官方文档 \(英文\)](#)
- [Redis 中文文档](#)

测试连接

`PING [message]` 命令可以测试和 `redis-server` 之间的连接是否正常。

```
Terminal
127.0.0.1:6379> PING
PONG
127.0.0.1:6379> PING "I love xixi" # 如果字符串中有空白符号，需要加引号
"I love xixi"
```

切换数据库

每个 Redis 实例都有 16 个数据库（编号 0 到 15），默认情况下，连接的是 0 号数据库。我们可以使用 `SELECT <index>` 命令切换数据库。

```
Terminal
127.0.0.1:6379> SELECT 8
OK
127.0.0.1:6379[8]>
```

查看数据库的大小

`DBSIZE` 可以查看数据库的大小（键的数目）。

```
Terminal
127.0.0.1:6379> DBSIZE
(integer) 0
127.0.0.1:6379> SET key1 hello
OK
127.0.0.1:6379> SET key2 world
OK
127.0.0.1:6379> DBSIZE
(integer) 2
```

清空数据库

`FLUSHDB [ASYNC]` 用于清空当前数据库，`FLUSHALL [ASYNC]` 用于清空所有数据库。默认情况下，两者均以同步方式执行；若指定 `ASYNC` 选项，则改为异步执行。简单来说，同步模式会阻塞 Redis 主线程直至清理完成，而异步模式则将清理工作交由后台线程处理，主线程可继续响应请求。

```
Terminal
127.0.0.1:6379> DBSIZE
(integer) 2
127.0.0.1:6379> FLUSHDB
OK
127.0.0.1:6379> DBSIZE
(integer) 0
```

关闭连接，退出 Redis

`quit` 和 `exit` 命令都可以关闭连接，退出 Redis 客户端。

```
Terminal
127.0.0.1:6379> exit
peanut@wd:~$
```

3.2 操作键 (Key) 的命令

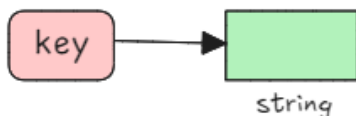
逻辑上，我们可以把一个 Redis 数据库看作是一张哈希表，其中键为字符串类型，值可以是各种各样的数据类型，比如：字符串 (string)、哈希表 (hash)、列表 (list)、集合 (set)、有序集合 (zset) 等。

命令	解释说明	示例
KEYS pattern	查找匹配模式的所有键	KEYS *
EXISTS key [key ...]	判断键是否存在	EXISTS user:1001
DEL key [key ...]	删除键值对	DEL user:1001
RENAME key newkey	对键重命名	RENAME user:1001 user:1002
MOVE key db	将键值对从一个数据库移动到另一个数据库	MOVE user:1001 1
TYPE key	查看键关联值的类型	TYPE user:1001
EXPIRE key seconds	设置键值对的过期时间 (单位为秒)	EXPIRE user:1001 3
PEXPIRE key milliseconds	设置键值对的过期时间 (单位为毫秒)	PEXPIRE user:1001 3000
TTL key	查看键的存活时间	TTL user:1001
PERSIST key	移除键的超时时间，变为永久存在的键	PERSIST user:1001
RANDOMKEY	从当前数据库中随机返回一个键	RANDOMKEY

3.3 字符串

接下来我们学习 Redis 5 大常用的数据类型：字符串 (string)、列表 (list)、哈希 (hash)、集合 (set) 和有序集合 (sorted set, zset)，它们可以满足工作中绝大多数场景的需求。

首先，我们来看字符串。字符串是 Redis 最基本的数据类型，也是最简单的值类型。它可以存储任意的二进制序列，比如：文本，序列化后的对象，二进制数组等。默认情况下，一个字符串类型的值最大可以存储 512MB 数据。



特点：
一个键(key)对应一个值(value)

常用命令

(1) 添加

命令	解释说明
SET key value [EX seconds PX milliseconds KEEPTTL] [NX XX]	设置键值对
SETEX key seconds value	设置键值对，并指定超时时间（秒）
PSETEX key milliseconds value	设置键值对，并指定超时时间（毫秒）
SETNX key value	仅当键不存在才设置，相当于 SET 加上 NX 参数
MSET key value [key value ...]	一次性设置多个键值对

我们重点讲讲 SET 命令，它也是字符串最核心的命令。

```
1 SET key value [EX seconds|PX milliseconds|KEEPTTL] [NX|XX]
```

参数解析：

- EX seconds: 设置过期时间 (秒)
- PX milliseconds: 设置过期时间 (毫秒)
- KEeptTL: 保持键的过期时间, 不加 KEeptTL 参数会移除过期时间
- NX: 仅当 key 不存在才设置 (Not eXists)
- XX: 仅当 key 存在时才设置

```

1 # 基本用法
2 SET name "Alice"
3
4 # 带过期时间 (10 秒后过期)
5 SET session:abc123 "session data" EX 10
6
7 # 获取分布式锁: 不存在时才设置
8 SET lock:order "locked" NX
9
10 # 组合使用
11 SET cache:item "value" NX EX 5

```

(2) 查找

命令	解释说明
GET key	获取字符串的值
MGET key [key ...]	批量获取
GETRANGE key start value	获取子串 [start, end], 索引从 0 开始, 支持负数索引
GETSET key value	返回旧值, 并设置新值 (原子操作)
STRLEN key	获取字符串的长度

(3) 修改

除了 SET 之外, 我们还可以通过下面命令修改字符串。

命令	解释说明
APPEND key value	追加字符串
SETRANGE key offset value	覆盖字符串: 从指定位置开始覆盖, 返回修改后的长度

(4) 数值操作

Redis 的字符串也可以作为数值使用, 当然前提是, 字符串能够解析为数值。

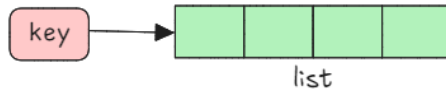
命令	解释说明
INCR key	将整数值加 1。如果 key 不存在, 会先初始化为 0, 再递增
INCRBY key increment	加上指定的值 (increment)
DECR key	将整数值减 1。如果 key 不存在, 会先初始化为 0, 再递减
DECRBY key decrement	减去指定的值 (decrement)
INCRBYFLOAT key increment	将浮点数加上一个给定的值 (increment)

提示

HELP @string 可以查看更多字符串相关的命令。

3.4 列表

Redis 的列表 (list) 是一种有序的字符串集合，它的核心特性是能够高效地在列表的头部或尾部执行插入和删除操作。这使得它非常灵活，可以用来实现队列、栈等多种数据结构。



特点：

- 有序的字符串集合
- 可以在 $O(1)$ 的时间复杂度内，在两端添加和删除

常用命令

(1) 添加

命令	解释说明
<code>LPUSH key element [element ...]</code>	将一个或多个值插入到列表头部 (左边)
<code>RPUSH key element [element ...]</code>	将一个或多个值插入到列表尾部 (右边)
<code>LINSERT key BEFORE AFTER pivot element</code>	在列表中指定的元素 pivot 之前或之后插入新值

(2) 查找

命令	解释说明
<code>LINDEX key index</code>	通过索引获取列表中的元素 (索引从 0 开始，并支持负数，-1 代表最后一个元素)
<code>LRANGE key start stop</code>	获取列表中指定范围内 [start, stop] 的元素
<code>LLEN key</code>	获取列表的长度 (元素个数)

(3) 修改

命令	解释说明
<code>LSET key index element</code>	通过索引设置列表元素的值

(4) 删除

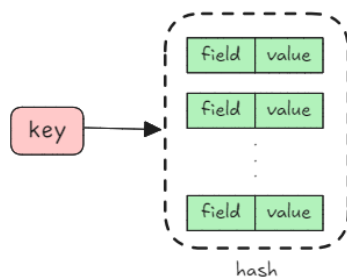
命令	解释说明
<code>LPOP key</code>	移除并返回列表的头部元素
<code>RPOP key</code>	移除并返回列表的尾部元素
<code>LREM key count element</code>	从列表中删除与 element 相等的元素；count 用于控制删除数量 (正数从头删，负数从尾删)
<code>LTRIM key start stop</code>	修剪列表，只保留指定区间内的元素，其余删除

提示

`HELP @list` 可以查看更多列表相关的命令。

3.5 哈希

Redis 的哈希 (hash) 类型非常适合存储对象，比如用户信息、商品详情等。它的内部结构是键值对集合，一个 key 对应多个 field-value 键值对。



特点：

1. 一个key对应多个键值对 (field-value)
2. 可以在 $O(1)$ 的时间复杂度内，对键值对进行增删改查

常用命令

(1) 添加

命令	解释说明
<code>HSET key field value [field value ...]</code>	添加或修改字段
<code>HSETNX key field value</code>	只在字段不存在时才设置，防止覆盖

(2) 查找

命令	解释说明
<code>HGET key field</code>	获取指定字段的值，不存在返回 (nil)
<code>HMGET key field [field ...]</code>	获取多个字段的值
<code>HGETALL key</code>	获取哈希中所有的字段和值
<code>HKEYS key</code>	获取所有字段名
<code>HVALS key</code>	获取所有字段的值
<code>HEXISTS key field</code>	检查某个字段是否存在：存在返回 1，不存在返回 0
<code>HLEN key</code>	获取哈希中字段的数量，哈希不存在则返回 0

(3) 修改

哈希没有专门的“修改”命令。如果字段已存在，`HSET key field value` 会更新字段的值。除此之外，哈希还提供了一些专门更新数值字段的命令：`HINCRBY` 和 `HINCRBYFLOAT`。

命令	解释说明
<code>HINCRBY key field increment</code>	为整数字段增加指定数值，若字段不存在，会先初始化为 0 再执行加法
<code>HINCRBYFLOAT key field increment</code>	为浮点数字段增加指定值，用法和 <code>HINCRBY</code> 一样，只是支持小数

(4) 删除

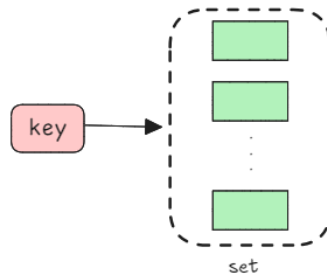
命令	解释说明
<code>HDEL key field [field ...]</code>	删除一个或多个字段，返回被成功删除的字段数量（不存在的字段忽略不计）

提示

`HELP @hash` 可以查看更多哈希相关的命令。

3.6 集合

集合是一个无序、唯一的字符串元素集合。它非常适合做标签系统、去重计数、共同好友计算等应用。



特点：

1. 集合中的成员(member)是无序的
2. 集合中的成员是唯一的
3. 可以在 $O(1)$ 的时间复杂度内，添加、查找和删除成员

常用命令

(1) 添加

命令	解释说明
<code>SADD key member [member ...]</code>	向集合中添加一个或多个元素

(2) 查找

命令	解释说明
<code>SMEMBERS key</code>	获取集合中所有的元素
<code>SCARD key</code>	获取集合的基数 (元素个数)
<code>SISMEMBER key member</code>	判断元素是否存在集合中，存在返回 1，不存在返回 0
<code>SRANDMEMBER key [count]</code>	随机返回元素。 <code>count</code> 为正，不放回抽样（不重复）； <code>count</code> 为负，放回抽样（可能重复）

(3) 修改

集合中没有直接修改元素的命令。要“修改”元素，只能：先删除旧的元素，再添加新的元素。

(4) 删除

命令	解释说明
<code>SREM key member [member ...]</code>	移除集合中一个或多个元素，忽略不存在的元素
<code>SPOP key [count]</code>	随机移除并返回 <code>count</code> 个元素。适合抽奖、随机分配等场景
<code>SMOVE source destination member</code>	将元素从 <code>source</code> 集合移动到 <code>destination</code> 集合

(5) 集合运算

这是集合类型最强大的功能。

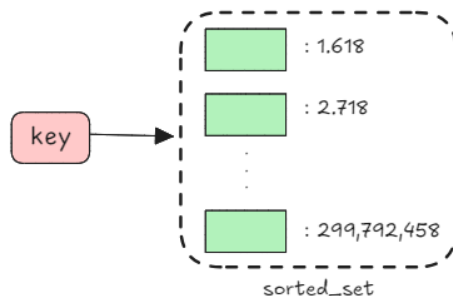
命令	解释说明
SINTER key [key ...]	求集合的交集：返回所有集合共同拥有的元素
SINTERSTORE destination key [key ...]	求集合的交集，将计算结果存入 destination 中
SUNION key [key ...]	求集合的并集：返回所有集合中出现过的元素
SUNIONSTORE destination key [key ...]	求集合的并集，将计算结果存入 destination 中
SDIFF key [key ...]	求集合的差集：返回在第一个集合中，但不在后续集合中的元素
SDIFFSTORE destination key [key ...]	求集合的差集，将计算结果存入 destination 中

提示

HELP @set 可以查看更多集合相关的命令。

3.7 有序集合

有序集合和集合很像，但每个元素都携带一个分数，并且会根据分数对元素排序。



特点：

1. 有序集合中的每个元素都会携带一个分数(score)
2. 元素会根据分数，从小到大排序

常用命令

(1) 添加

命令	解释说明
ZADD key [NX XX] [CH] [INCR] score member [score member ...]	向集合中添加一个或多个元素

添加一个或多个元素，已存在的元素会更新分数（可通过参数控制）；返回新增元素的数量（若使用 **CH** 参数，则返回被修改元素总数）。

参数讲解：

- **NX**：仅添加元素，不更新已有元素。
- **XX**：仅更新已有元素，不添加元素。
- **CH**：返回值从新增数量改为被修改元素总数（新增+更新）。
- **INCR**：对元素分数做增量操作（只能作用于一个元素，返回新分数）。

(2) 查找

命令	解释说明
ZCARD key	获取有序集合的基数 (元素个数)。
ZSCORE key member	获取元素关联的分数。
ZCOUNT key min max	统计 [min, max] 区间的元素个数。
ZRANGE key start stop [WITHSCORES]	获取排名区间 [start, stop] 内的元素。
ZRANGEBYSCORE key min max [WITHSCORES] [LIMIT offset count]	返回指定分数区间内的元素。
ZRANK key member	获取元素的正向排名。
ZREVRANK key member	获取元素的逆向排名。

我们重点讲讲 ZRANGEBYSCORE key min max [WITHSCORES] [LIMIT offset count]，这个命令可配合 LIMIT 进行分页查询。

```

Terminal

# 添加测试数据
127.0.0.1:6379> ZADD scores 100 Alice
127.0.0.1:6379> ZADD scores 85 Bob
127.0.0.1:6379> ZADD scores 92 Charlie
127.0.0.1:6379> ZADD scores 78 David
127.0.0.1:6379> ZADD scores 95 Eve
127.0.0.1:6379> ZADD scores 88 Frank
# 按分数从低到高排序，第一页 (每页两条)
127.0.0.1:6379> ZRANGEBYSCORE scores -inf +inf LIMIT 0 2
1) "David"
2) "Bob"
# 第二页
127.0.0.1:6379> ZRANGEBYSCORE scores -inf +inf LIMIT 2 2
1) "Frank"
2) "Charlie"
# 第三页
127.0.0.1:6379> ZRANGEBYSCORE scores -inf +inf LIMIT 4 2
1) "Eve"
2) "Alice"

```

(3) 修改

ZADD 可以对已有元素进行更新，除此之外，有序集合还提供了更方便使用的 ZINCRBY 命令。

命令	解释说明
ZINCRBY key increment member	将 member 元素的分数增加 increment。

(4) 删除

命令	解释说明
ZREM key member [member ...]	删除指定元素，返回实际被删除的数量。
ZPOPMAX key [count]	弹出分数最高的一个或多个元素，返回 [member, score] 的列表。
ZPOPMIN key [count]	弹出分数最低的元素，用法同 ZPOPMAX。
ZREMRANGEBYSCORE key min max	按分数范围删除，返回删除数量。
ZREMRANGEBYRANK key start stop	按排名范围删除 (最低分排名为 0)；返回删除数量。

提示

HELP @sorted_set 可以查看更多集合相关的命令。

4 Redis 事务

很多同学对 Redis 事务都有一个错误的认知，认为 Redis 事务和关系型数据库（如 MySQL）的事务差不多，都提供了 ACID 的保障。但实际上，它们有本质的区别，Redis 事务的本质是命令批量执行，它不提供完整的 ACID 保障。

4.1 相关命令

命令	解释说明
MULTI	开启事务，后续命令不会立即执行，而是放入队列。
EXEC	执行事务，事务中的命令按入队的顺序执行。
DISCARD	取消事务，清空事务队列，并退出事务状态。
WATCH key [key ...]	在 MULTI 之前执行，监视一个或多个键。如果这些键在 EXEC 之前被修改，则事务执行失败，提供“乐观锁”的功能。
UNWATCH	取消所有被 WATCH 的键。

```
Terminal

# 1. 监视一个键（乐观锁）
127.0.0.1:6379> WATCH mylock
OK
# 2. 开启事务
127.0.0.1:6379> MULTI
OK
# 3. 输入多条命令（此时不会执行，仅入队）
127.0.0.1:6379> SET newkey newvalue
QUEUED
127.0.0.1:6379> INCR counter
QUEUED
# 4. 执行事务
127.0.0.1:6379> EXEC
1) OK
2) (integer) 1
```

如果不想执行事务了，用 DISCARD 代替 EXEC。

4.2 执行流程

Redis 事务是分三个阶段执行的：

1. 命令入队 (Queueing)
 - a) 客户端发送 MULTI，服务端返回 OK，客户端状态变为“事务中”。
 - b) 后续所有非 EXEC/DISCARD/WATCH 的命令，不会被立即执行，而是被服务器放入该客户端的事务队列中，并返回 QUEUED。
2. 执行 (Executing)

客户端发送 EXEC，Redis 会按顺序依次执行事务队列中的所有命令。
3. 结果返回 (Response)

所有命令的执行结果会按顺序打包成一个数组，一次性返回给客户端。

4.3 Redis 事务的 ACID 特性（与关系型数据库对比）

特性	Redis 事务表现	解释说明
原子性（Atomicity）	不支持	Redis 事务不支持回滚。如果事务中某条命令执行出错，其他命令依然会继续执行，不会回滚。
一致性（Consistency）	部分支持	在无宕机、无非法数据的情况下，事务执行后数据是符合语义的。但无法保证业务逻辑一致性（比如转账）。
隔离性（Isolation）	完全支持	Redis 是单线程的，事务执行期间不会被其他操作打断，隔离性最高（SERIALIZABLE）。
持久性（Durability）	取决于配置	仅当开启 AOF 且设置 <code>appendfsync=always</code> 时，事务结果才会立即落盘。否则数据可能丢失。

5 Redis 的持久化机制

如果有人问：“你会把 Redis 用在什么业务场景下？” 很大概率我们会回答：“把它当作缓存使用。” 没错，缓存是 Redis 的一个主要使用场景。但是，缓存也有一个绝对不能忽略的问题：一旦服务器宕机，缓存（内存）中的数据将全部丢失。

你很可能觉得这无关紧要，毕竟我们可以从后端数据库恢复这些数据。但是从数据库恢复数据，存在两个弊端：

1. 会给数据库带来巨大的访问压力。
2. 从慢速数据库中慢慢恢复，性能肯定比不上直接从 Redis 中读取，从而导致应用程序响应变慢。

所以，对 Redis 来说，实现数据的持久化，避免从后端数据库中进行恢复，是至关重要的。目前，Redis 的持久化有两大机制，即 AOF（Append Only File）日志和 RDB 快照。

5.1 RDB 快照

RDB（Redis Database）是 Redis 默认的一种持久化方式。Redis 会在某个时间点给内存中的全量数据生成一份快照，并以二进制 .rdb 文件的形式保存 to 磁盘中。

Redis 重启时，会自动加载 .rdb 文件快速恢复数据。与 AOF 记录命令不同，RDB 直接保存数据本身，因此恢复速度极快，非常适合用于数据备份和灾难恢复。

触发方式

RDB 持久化有两种触发方式：

- 手动触发：SAVE 或 BGSAVE 命令。
 - SAVE：由主进程执行（同步），会阻塞所有客户端请求，直到快照完成。在生产环境应避免使用。
 - BGSAVE：推荐方式。主进程会 fork 出一个子进程，由子进程在后台生成 RDB 文件（异步），主进程继续处理请求，不会阻塞服务。
- 自动触发：通过 redis.conf 中的 save 配置选项实现。

```
1 # 格式: save <秒数> <修改次数>
2 save 900 1      # 900 秒（15 分钟）内至少有 1 个 key 被修改
3 save 300 10     # 300 秒（5 分钟）内至少有 10 个 key 被修改
4 save 60 10000   # 60 秒（1 分钟）内至少有 10000 个 key 被修改
```

只要满足任一条件，Redis 就会自动触发 BGSAVE。此外，主从复制、正常关闭服务器等场景也会触发 RDB 生成。

BGSAVE 的原理

BGSAVE 的完整工作流程是 RDB 机制的精华所在，它巧妙地利用了操作系统的写时复制（Copy-On-Write）技术，实现了高效的快照生成。

1. Redis 主进程 fork 出一个子进程。此时，子进程会获得主进程内存页表的副本（并非实际物理内存的副本）。
2. 子进程根据页表，将内存中的数据序列化为二进制格式，写入一个临时的 RDB 文件（如 temp-<pid>.rdb）。
3. 在子进程生成快照期间，主进程依然可以正常处理写请求。此时，操作系统层面的写时复制技术开始发挥作用：
 - 对于读操作，主进程和子进程共享同一份物理内存，没有任何开销。
 - 对于写操作，操作系统会将受影响的物理内存页复制一份（“复制”的粒度是内存页，通常为 4KB），然后主进程在新副本上进行修改，而子进程看到的仍然是旧副本——即 fork 时刻的数据快照。
4. 子进程生成完成临时文件后，会通知主进程。主进程会原子地将临时文件重命名为正式的 dump.rdb 文件，完成本次快照。

这个机制确保了子进程拥有一个一致性的数据视图，同时主进程的写操作不会受到阻塞。

配置参数

```
1 # 1. 自动触发条件（生产环境根据可接受的数据丢失量设置）
2 save 900 1      # 15 分钟内至少改动 1 次（长时间内只要改动过一次，就保存）
3 save 300 10     # 5 分钟内至少改动 10 次
4 save 60 10000   # 60 秒内至少改动 10000 次（高频写入场景下，防止频繁保存影响性能）
5
6 # 2. RDB 文件名
7 dbfilename dump.rdb
8
9 # 3. RDB 文件保存目录
10 dir /var/lib/redis
11
12 # 4. 是否压缩 RDB 文件（可以节省空间，但消耗 CPU）
13 rdbcompression yes
14
15 # 5. 是否对 RDB 文件进行校验（防止损坏）
16 rdbchecksum yes
17
18 # 6. 禁止自动保存（如果你只用 AOF，则注释所有 save 行）
19 # save ""
```

优缺点

优点：

- RDB 文件是某个时间节点的快照，默认使用 LZF 算法进行压缩，压缩后的文件体积远远小于内存大小，适用于备份、全量复制等场景。
- Redis 使用 RDB 文件恢复数据的方式，远远快于 AOF 方式。

缺点：

- RDB 方式实时性不够，无法做到秒级的持久化，数据安全性不如 AOF。
- 每次调用 bgsave 都需要 fork 子进程，fork 子进程属于重量级操作，频繁执行成本较高。
- RDB 文件是二进制的，没有可读性，AOF 文件在了解其结构的情况下可以手动修改或者补全。

针对 RDB 不适合实时持久化的问题，Redis 提供了 AOF 持久化方式。

5.2 AOF 日志

AOF（Append Only File）是 Redis 的一种高可靠性持久化机制，通过记录所有写操作命令并在重启时重放这些命令来恢复数据，为应对服务器意外宕机提供了比 RDB 更精细的数据保护。

原理

1. 追加命令到缓冲区

当 AOF 功能开启后，Redis 服务器每执行完一个写命令（如 SET、LPUSH），不会立刻写入磁盘，而是先将命令以 RESP 协议格式追加到内存中的一个名字叫 `aof_buf` 缓冲区的末尾。

2. 文件写入与同步

这是平衡数据安全性和性能的关键。Redis 有三个写回策略，你可以根据业务对数据丢失的容忍度进行相应的选择。

策略 (appendfsync)	工作机制	优缺点
always	每个写命令执行后，立即同步写入磁盘	优点： 安全性最高，最多只丢失一个命令的数据； 缺点： 性能最差，每次写入都要等待磁盘 I/O
everysec(默认)	每秒执行一次同步操作，由后台线程负责	优点： 性能和安全性平衡，最多丢失一秒内的数据，是绝大多数生产环境的推荐配置； 缺点： 若发生宕机，可能丢失最近一秒的数据
no	由操作系统决定何时写回	优点： 性能最高，因为 Redis 无需等待磁盘操作； 缺点： 安全性最低，宕机时可能丢失大量数据

AOF 重写：为臃肿的日志文件“瘦身”

随着服务器运行时间变长，AOF 文件会不断增长，这不仅占用磁盘，更会导致 Redis 重启时恢复时间漫长。AOF 重写机制就是为了解决这个问题的。

它的核心思想不是去修改旧的 AOF 文件，而是基于当前内存中的数据，重新生成一份新的 AOF 文件，其中只包含恢复当前数据集所需的最少命令集合。例如，对一个 key 执行了 100 次 SET，重写后可能只保留最后那条 SET 命令。并且整个重写过程是安全的，不会阻塞主进程服务。

AOF 重写有两种触发方式：

- 手动触发：BGREWRITEAOF。
- 自动触发：由 Redis 根据配置（`auto-aof-rewrite-percentage` 和 `auto-aof-rewrite-min-size`）自动触发。

AOF 重写的原理如下：

1. Redis 主进程会 fork 出一个子进程，由子进程在后台执行重写任务，而主进程继续处理客户端请求，互不干扰。
2. 在子进程重写期间，主进程收到的所有新的写命令会被同时记录到一个“重写缓冲区”中。
3. 待子进程完成新 AOF 文件的生成后，主进程会将这些缓冲区的增量命令追加到新文件末尾，这样就保证了数据完整性，然后原子地替换旧的 AOF 文件。

配置参数

```
1 # 1. 开启 AOF
2 appendonly yes
3
4 # 2. 同步策略：每秒同步
5 # appendfsync always
6 appendfsync everysec
7 # appendfsync no
8
```

```

9 # 3. 自动重写条件: 相比上次增长 100% 且不小于 64MB
10 auto-aof-rewrite-percentage 100
11 auto-aof-rewrite-min-size 64mb
12
13 # 4. 开启混合持久化 (Redis 4.0+ 特性)
14 aof-use-rdb-preamble yes
15
16 # 5. 重写期间暂停 fsync, 提升性能
17 no-appendfsync-on-rewrite yes

```

6 缓存问题：缓存穿透、缓存击穿、缓存雪崩

Redis 最常用的一个场景就是作为缓存，这一章节主要探讨 Redis 作为缓存，在实践中可能会遇到有哪些问题？比如缓存穿透、缓存击穿、缓存雪崩等。这些也是面试常问的一些问题。

在高并发业务场景中，数据库通常会成为整个系统链路中最薄弱的环节。因此，我们会引入 Redis 作为缓存层，使请求先访问 Redis，而不是直接打到 MySQL 等关系型数据库上，从而有效减轻数据库的压力。



系统引入缓存层后，虽能提升性能，但也带来了新的问题：

- 缓存穿透
- 缓存击穿
- 缓存雪崩

6.1 缓存穿透

缓存穿透是指：客户端请求的数据既不存在于缓存中，也不存在于数据库中；请求会直接穿透缓存层，打到数据库上。由于数据库里也没有这条数据，所以无法回写缓存。结果是，每一次请求都会直接访问数据库。



如果这类请求量很大（例如恶意攻击），数据库会承受巨大的压力，甚至可能被压垮。

示例

假设我们的系统里，用户 ID 是从 1 到 10000 的正整数。恶意攻击者发起大量请求，访问：

```

/user?id=-1
/user?id=-2
/user?id=999999999

```

这些 ID 在数据库中根本不存在，缓存中当然也没有。于是每个请求都会穿透到数据库，数据库每次都返回“查无此数据”，然后也不会写入缓存。结果就是，数据库被无效查询反复冲击。

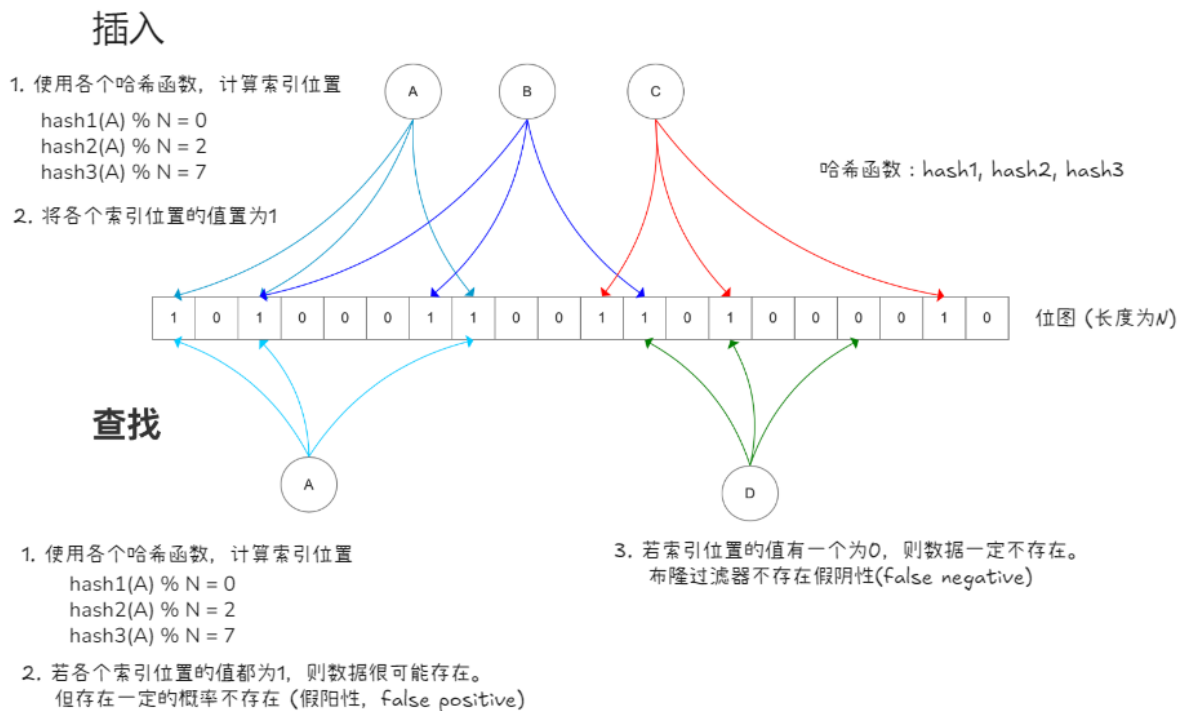
解决方案

方案 1：缓存空值或默认值（最常用）

当数据库查询不到数据时，将一个空值或默认值（比如 "NULL" 或 0）写入缓存，并设置一个较短的过期时间（如 30 秒~5 分钟）。这样，后续的请求会直接从 Redis 中读取空值或默认值；有效地避免了把大量请求发送给数据库。

方案 2：布隆过滤器（Bloom Filter）

在缓存和数据库之前，增加一个布隆过滤器，用来判断一个数据是否可能存在于数据库中。布隆过滤器的本质就是位图+一组哈希函数，原理如下图所示：



- 如果布隆过滤器说“不存在” → 一定不存在，直接返回。
- 如果布隆过滤器说“存在” → 可能存在，放行去查数据库。

方案 3：请求参数校验

在入口层（如 API 网关）对请求参数进行合法性校验。

例如：

- ID 必须 > 0 且 1000000 。
- 手机号必须是 11 位数字。
- 时间范围不能超过 31 天。

这种方法虽然简单，但能过滤掉一大批明显的非法请求。

生产中的组合方案

在实际生产中，大多数公司会组合使用这几种方案：

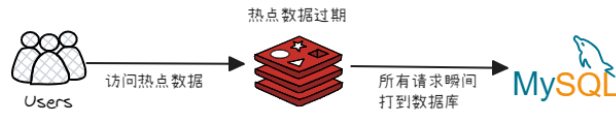
- 第一层：参数校验（过滤明显非法请求）。
- 第二层：布隆过滤器（拦截绝大多数不存在的 key）。

- 第三层：缓存空对象（兜底，处理布隆过滤器的假阳性情况）。

如果对数据一致性要求不是很苛刻，很多场景下直接用参数校验+缓存空对象就足够了，实现简单且可靠。

6.2 缓存击穿

缓存击穿是指：某个热点数据的缓存突然过期失效，此时有大量并发请求同时访问这个数据，由于缓存中没有，所有请求都会瞬间打到数据库上，造成数据库压力骤增。



示例

假设有一个系统，缓存了一个爆款商品的信息：

商品 ID: 123456

缓存过期时间：中午 12 点

这个商品平时每秒被访问 5 万次

中午 12 点那一刻：缓存过期了

第 1 个请求来了：缓存没有 --> 去数据库查询 --> 回写缓存

但问题是：在同一秒内，有 5 万个请求同时到达

结果就是：5 万个请求全部发现缓存为空，全部去查询数据库。

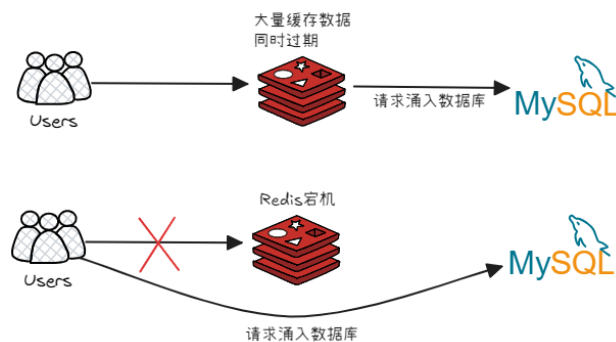
数据库瞬间压力飙升：从平时的几百~几千的 QPS 飙升到 5 万 QPS，很可能直接被打崩。

解决方案

缓存击穿是由热点数据过期导致的，那么最直接有效的解决方案就是：对于热点数据，我们就不设置过期时间了。

6.3 缓存雪崩

缓存雪崩是指：大量缓存数据在同一时间段内集中过期，或者 Redis 服务本身宕机，导致大量请求无法命中缓存，全部涌入数据库，造成数据库压力骤增甚至崩溃的现象。



示例

假设有一个系统，要做促销活动，活动维持一个小时：

活动开始前：批量设置了活动的缓存数据，过期时间为 1 小时。

1 小时候，活动结束，缓存数据同时失效。

但用户并不会立刻停止访问，所有请求全部涌入数据库，数据库压力瞬间飙升。

解决方案

方案 1：过期时间打散（加随机值）

这是最直接的防“同时失效”手段。在设置过期时间时，给基础 TTL 之上加一个随机偏移量（如 `TTL + random(0, 300)` 秒），让缓存均匀地失效。

方案 2：搭建 Redis 高可用集群

确保 Redis 服务的高可用，避免出现单点故障。

方案 3：熔断降级

当访问 Redis 的异常比例达到阈值时，直接熔断缓存访问，返回兜底数据（如预设的空值或默认页面，降级处理），而不是让请求涌入到数据库。

方案 4：限流

当缓存雪崩发生时，限流是保护数据库的最后一道防线。它的核心思想是：既然流量洪峰无法避免，那就主动“丢弃”部分请求，牺牲少量用户体验，换取整体系统的存活。

注意

服务熔断、服务降级、请求限流这些方法都是属于“有损”方案，会对业务带来负面的影响，从而影响用户的体验。

7 Redis++

Redis++（也写作 `redis-plus-plus`）是一个基于 C++11/17 的、功能强大的 Redis 客户端。它是为了能够在 C++ 项目中方便地连接和操作 Redis 数据库而设计的。

7.1 安装

```
1 # 先安装 hiredis, 这是因为 redis-plus-plus 是基于 hiredis 的
2 tar xzvf hiredis-1.3.0.tar.gz
3 cd hiredis-1.3.0/
4 make
5 sudo make install
6 sudo ldconfig
7 # 安装 redis-plus-plus
8 tar xzvf redis-plus-plus-1.3.15.tar.gz
9 cd redis-plus-plus-1.3.15/
10 mkdir build && cd build
11 cmake ..
12 make
13 sudo make install
14 sudo ldconfig
```

安装成功后，头文件会放在 `/usr/local/include/sw` 目录下，而库文件放在 `/usr/local/lib` 目录下。

7.2 示例代码

redis++使用起来很方便，请看下面的示例代码（见 examples/redis_demo1.cc）：

redis_demo1.cc

```
1  #include <iostream>
2  #include <sw/redis++/redis++.h>
3
4  using namespace std;
5  using namespace sw::redis;
6
7  int main()
8  {
9      // 创建一个 Redis 对象，连接 redis-server
10     auto redis = Redis("tcp://127.0.0.1:6379");
11
12     // ===== STRING 字符串命令 =====
13
14     redis.set("key", "val");
15
16     // 获取键对应的值，返回 OptionalString 类型
17     // OptionalString 可能包含值也可能为空
18     auto val = redis.get("key");
19     if (val) {
20         // 如果包含值，解引用获取 string 类型的值
21         cout << *val << endl;
22     } else {
23         // 否则键不存在
24     }
25
26     // ===== LIST 列表命令 =====
27
28     // 使用 vector<string> 向列表中添加元素
29     vector<string> vec = { "a", "b", "c" };
30     redis.rpush("list", vec.begin(), vec.end());
31
32     // 使用 initializer_list 向列表中添加元素
33     redis.rpush("list", { "a", "b", "c" });
34
35     // 从 Redis LIST 读取数据到 vector<string>
36     vec.clear();
37     redis.lrange("list", 0, -1, back_inserter(vec)); // LRANGE 0 -1 表示获取所有元素
38
39     // ===== HASH 哈希命令 =====
40
41     // 设置哈希表字段：hash 中的 field 字段值为 "val"
42     redis.hset("hash", "field", "val");
43
44     // 另一种设置方式：使用 pair 键值对
45     redis.hset("hash", make_pair("field", "val"));
46
47     // 使用 unordered_map 批量设置哈希表
48     unordered_map<string, string> m = {
49         { "field1", "val1" },
50         { "field2", "val2" }
51     };
52     redis.hset("hash", m.begin(), m.end()); // HSET 批量设置多个字段
53
54     // 从哈希中读取数据到 unordered_map
55     m.clear();
56     redis.hgetall("hash", inserter(m, m.begin())); // HGETALL 获取所有字段和值
57
58     // 只获取特定字段的值
59     // 注意：字段可能不存在，所以需要 OptionalString 处理
60     vector<OptionalString> vals;
61     redis.hmget("hash", { "field1", "field2" }, back_inserter(vals)); // HMGET 获取多个字段
62
63     // ===== SET 集合命令 =====
64
65     // 向集合添加单个元素
66     redis.sadd("set", "m1");
67
68     // 使用 unordered_set 批量添加元素到集合
69     unordered_set<string> set = { "m2", "m3" };
```

```

70 redis.sadd("set", set.begin(), set.end());
71
72 // 使用 initializer_list 批量添加元素
73 redis.sadd("set", { "m2", "m3" });
74
75 // 从集合中读取数据到 unordered_set
76 set.clear();
77 redis.smembers("set", inserter(set, set.begin())); // SMEMBERS 获取集合所有成员
78
79 // 检查元素是否存在于集合中
80 if (redis.sismember("set", "m1")) { // SISMEMBER 判断成员是否存在
81     cout << "m1 exists" << endl;
82 } // 否则不存在
83
84 // ===== SORTED SET 有序集合命令 =====
85
86 // 向有序集合添加单个元素 (成员名和分数)
87 redis.zadd("sorted_set", "m1", 1.3);
88
89 // 使用 unordered_map 批量添加元素到有序集合
90 unordered_map<string, double> scores = {
91     { "m2", 2.3 },
92     { "m3", 4.5 }
93 };
94 redis.zadd("sorted_set", scores.begin(), scores.end());
95
96 // 从有序集合读取数据到 vector<pair<string, double>>
97 // 注意: zrangebyscore 返回的结果是有序的, 如果存储到 unordered_map 会丢失顺序
98 vector<pair<string, double>> zset_result;
99 redis.zrangebyscore("sorted_set",
100     UnboundedInterval<double> {}, // 区间: (-inf, +inf), 即所有元素
101     back_inserter(zset_result));
102
103 // 只获取成员名 (不获取分数):
104 // 传入 vector<string> 类型的插入迭代器作为输出参数
105 vector<string> without_score;
106 redis.zrangebyscore("sorted_set",
107     BoundedInterval<double>(1.5, 3.4, BoundType::CLOSED), // 区间: [1.5, 3.4], 闭区间
108     back_inserter(without_score));
109
110 // 同时获取成员名和分数:
111 // 传入 vector<pair<string, double>> 的 back_inserter
112 vector<pair<string, double>> with_score;
113 redis.zrangebyscore("sorted_set",
114     BoundedInterval<double>(1.5, 3.4, BoundType::LEFT_OPEN), // 区间: (1.5, 3.4], 左开右闭
115     back_inserter(with_score));
116
117 // ===== Transaction 事务 =====
118
119 // 创建一个事务对象
120 auto tx = redis.transaction();
121
122 // 在事务中运行多个命令, 并获得所有响应
123 auto tx_replies = tx.incr("num0") // 对 num0 自增 1
124     .incr("num1") // 对 num1 自增 1
125     .mget({ "num0", "num1" }) // 批量获取 num0 和 num1 的值
126     .exec(); // 执行事务
127
128 // 解析事务响应, 通过响应类型和索引获取结果
129 // 第 0 个命令 (incr) 的返回值是 long long 类型
130 auto incr_result0 = tx_replies.get<long long>(0);
131
132 // 第 1 个命令 (incr) 的返回值是 long long 类型
133 auto incr_result1 = tx_replies.get<long long>(1);
134
135 // 第 2 个命令 (mget) 的返回值是 vector<OptionalString> 类型
136 vector<OptionalString> mget_cmd_result;
137 tx_replies.get(2, back_inserter(mget_cmd_result));
138
139 return 0;
140 }

```

使用 `g++ redis_demo1.cc -lredis++ -lhiredis` 即可编译、链接、生成可执行程序。